

DEVOPS MASTER COURSE

Volume 9F

Enterprise Automation Projects & Interview Mastery

*Advanced Troubleshooting, Certifications, System Design, and the Final Terraform-
Ansible Capstone*

Chapters 56-70

Production objective: demonstrate that you can design, build, operate, troubleshoot, secure, and explain an enterprise automation platform - not merely write isolated Terraform configurations or Ansible playbooks.

2026 Edition

About Volume 9F

Part F is the graduation volume for Terraform and Ansible infrastructure automation. It converts the language, platform, governance, delivery, and operational skills from Parts A-E into portfolio-grade projects and incident-response capability. Every architecture is evaluated through ownership boundaries, blast radius, identity, evidence, rollback, business continuity, and supportability.

The projects are intentionally vendor-neutral. They can be implemented on AWS, Azure, Google Cloud, VMware, OpenStack, Kubernetes, network devices, Windows, Linux, SaaS APIs, or a controlled local laboratory. The goal is to prove transferable engineering judgment rather than dependency on one provider console.

Certification coverage is a preparation framework, not an exam dump. Exam names, versions, delivery models, and objectives change. Verify the current official guide before registration and use the laboratories in this volume to build durable skill beyond a particular blueprint.

Volume 9 Structure and Progress

Part	Primary Focus	Status
9A	Terraform foundations, state, modules, import, and team workflow	Complete
9B	Advanced Terraform testing, policy, HCP operations, security, and Stacks	Complete
9C	Ansible foundations, inventory, playbooks, templates, handlers, and Vault	Complete
9D	Advanced Ansible roles, collections, execution environments, controller, Windows, networks, and testing	Complete
9E	Integrated Terraform-Ansible delivery, GitOps, supply-chain security, governance, and observability	Complete
9F	Enterprise projects, incident troubleshooting, certifications, interviews, and final capstone	Current volume

Part F Chapter Map

Chapter	Title
56	Enterprise Automation Architecture and Operating Model
57	Portfolio Discovery, Project Selection, and Value Measurement
58	Multi-Account and Multi-Cloud Landing-Zone Automation Project
59	Kubernetes and Application-Platform Automation Project
60	Data, Middleware, and Stateful-Service Automation Project
61	Brownfield Discovery, Import, Refactoring, and Standardization
62	Disaster Recovery, Rebuild Automation, and Continuity Testing
63	Advanced Terraform Troubleshooting and State Incident Response
64	Advanced Ansible Troubleshooting and Controller Incident Response
65	Integrated Pipeline, Identity, and Supply-Chain Incident Response
66	Performance, Scale, Reliability, and Capacity Engineering
67	Automation Platform Product Management and Governance
68	Terraform and Ansible Certification Mastery
69	Technical Interviews and Automation System Design
70	Final Enterprise Automation Factory Capstone

Enterprise Lab Safety and Evidence Standard

- Run destructive exercises only in disposable or explicitly approved environments. Apply budget limits, quotas, TTL tags, automatic cleanup, and deny-by-default credentials.
- Create an incident or change record for every lab. Preserve configuration commit, dependency lock files, plan or job artifact digest, approval identity, execution logs, verification results, and cleanup evidence.
- Use short-lived workload identity or controller-managed credentials. Do not place long-lived cloud keys, passwords, SSH private keys, or vault passwords in repositories, pipeline variables, screenshots, or support bundles.
- Back up Terraform state before manual state operations. Treat force-unlock, state push, state rm, targeted apply, and controller relaunch as controlled recovery actions requiring peer review.
- Use independent verification after deployment. A successful Terraform apply or Ansible job is evidence of tool completion, not proof that the service is secure, reachable, recoverable, or meeting an SLO.
- Sanitize all evidence before sharing. State, plans, controller events, callback output, inventory, and verbose logs can contain identifiers and sensitive data even when display masking is enabled.

Chapter 56: Enterprise Automation Architecture and Operating Model

Enterprise automation is a socio-technical operating system. The code is only one component; the complete platform also includes ownership, identity, state, inventories, execution infrastructure, approvals, evidence, support, and recovery. This chapter establishes the architecture review method used throughout Part F.

Learning Objectives

- Define an automation platform as a product with explicit consumers and service levels.
- Separate control-plane, execution-plane, and managed-resource responsibilities.
- Choose state, inventory, credential, and artifact boundaries that contain blast radius.
- Produce architecture decision records that connect technical choices to business risk.

Core Concepts

Platform product: Treat reusable modules, roles, collections, pipelines, policies, and controller templates as supported products with owners, versions, documentation, and deprecation paths.

Control-plane boundaries: Terraform state services, automation controllers, source repositories, registries, identity brokers, secrets systems, policy engines, and logging backends require stronger isolation than ordinary workloads because compromise enables broad downstream change.

Execution-plane design: Runners and execution nodes should be ephemeral where practical, restricted to required networks, pinned to verified toolchains, and unable to read unrelated states or inventories.

Ownership model: Every mutable property has one authoritative owner. Terraform usually owns durable resource lifecycle; Ansible owns operating-system and application convergence; specialized operators may own Kubernetes-native or database lifecycle.

Service levels: Define availability, queue latency, plan turnaround, deployment success, recovery time, drift-response time, and support hours for the automation service itself.

Architecture evidence: A credible design includes context, trust boundaries, data flows, identity paths, failure modes, rollback choices, operational dashboards, and test results - not only a component diagram.

Reference Configuration or Command Pattern

```
# architecture decision record excerpt
adr_id: ADR-056
decision: separate-state-per-service-environment
context:
  blast_radius: high
  team_ownership: federated
controls:
  - remote_state_locking
  - workload_identity
  - peer_reviewed_apply
  - independent_post_deploy_verification
recovery:
  rto_minutes: 60
evidence: [state_backup, run_log, verification_report]
```

Production Checklist

- Platform consumers and support owners are named.
- Trust boundaries and credential paths are diagrammed.

- State and inventory boundaries align with failure domains.
- Each managed property has one authoritative owner.
- Platform SLOs and recovery objectives are measurable.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Teams bypass the platform	Golden paths are slower than direct console changes	Measure friction, simplify safe defaults, and reserve exceptions for documented cases	Adoption metrics, bypass audit, user interviews
One failure blocks all automation	Shared control-plane component has no isolation or queue partitioning	Introduce tenant boundaries, redundant workers, and workload-specific quotas	Queue metrics, dependency map, failover test
No one owns failed deployments	Responsibility stops at tool execution	Define service ownership, escalation, and post-deployment verification	RACI, on-call route, incident timeline
Architecture reviews become diagrams only	Nonfunctional requirements are not testable	Attach SLOs, threat scenarios, recovery drills, and evidence requirements	ADR, test plan, game-day report

Hands-on Lab 56: Create an Enterprise Automation Architecture Review

Prerequisites

- A repository representing a Terraform-Ansible delivery workflow.
- A diagramming tool and an architecture decision record template.
- A disposable target environment or realistic design scenario.

Implementation

1. Map users, repositories, state backends, inventories, controllers, registries, identity providers, secret stores, policy services, and target systems.
2. Draw data flows and trust boundaries, including who can trigger, approve, execute, and read evidence.
3. Assign one owner and recovery procedure to each platform component.
4. Define at least five platform SLOs and three failure-domain boundaries.
5. Run a tabletop scenario for repository compromise, state-backend outage, and runner credential misuse.
6. Publish the architecture review with prioritized remediation actions and acceptance criteria.

Validation Evidence

- Every credential path terminates at a named identity and authorization control.
- No high-impact component lacks backup, failover, or rebuild procedure.
- The review includes measurable SLOs and tested failure scenarios.
- Remediation items have owners, deadlines, and evidence of closure.

Cleanup

- Remove temporary diagrams containing real identifiers from public locations.
- Revoke any test credentials and archive sanitized evidence.

Interview Questions and Model Answers

1. Why call automation a platform product?

Because teams depend on its reliability, interfaces, support, security, and evolution. Treating it as scripts hides operational obligations.

2. What is the highest-risk automation asset?

Usually the control plane or credential broker, because compromise can authorize changes across many environments even without direct workload access.

3. How should state boundaries be chosen?

Align them with ownership, lifecycle, privilege, rate of change, and failure blast radius rather than convenience alone.

4. What proves an architecture is operable?

Successful recovery exercises, observable SLOs, clear ownership, and evidence that normal and failure workflows can be executed safely.

Chapter Summary

- Automation architecture includes people, process, identity, state, execution, evidence, and recovery.
- Control-plane compromise has outsized blast radius.
- Boundaries should follow ownership and failure domains.
- Architecture decisions must be validated through exercises, not presentation alone.

Chapter 57: Portfolio Discovery, Project Selection, and Value Measurement

Automation programs fail when they optimize for the number of scripts instead of measurable operational outcomes. A disciplined portfolio process identifies repeated pain, estimates risk and value, chooses the correct automation level, and retires work that does not produce durable benefit.

Learning Objectives

- Build an automation opportunity inventory from operational evidence.
- Prioritize projects using value, risk, frequency, and feasibility.
- Define baselines and outcome metrics before implementation.
- Create an adoption and deprecation plan for every automation product.

Core Concepts

Opportunity discovery: Use incident records, ticket queues, change failures, toil surveys, audit findings, environment lead time, and capacity constraints to identify repeatable work.

Value model: Estimate hours avoided, lead-time reduction, availability improvement, risk reduction, evidence quality, and opportunity cost. Avoid claiming savings without a measured baseline.

Automation suitability: Prefer deterministic, repeatable, high-volume, well-understood processes. Delay automation when inputs are unstable, ownership is unclear, or the process itself is defective.

Risk-adjusted priority: A frequent task with moderate savings may outrank a rare high-effort task. Include blast radius, reversibility, dependency maturity, and availability of test environments.

Adoption design: Documentation, migration assistance, compatibility, observability, and responsive support determine adoption more than technical elegance.

Lifecycle management: Track versions, consumers, exceptions, maintenance cost, and retirement dates. An automation portfolio accumulates risk when obsolete paths remain executable.

Reference Configuration or Command Pattern

```
# weighted opportunity score (example)
score = (frequency * minutes_saved * users)
      + risk_reduction_points
      + compliance_evidence_points
      - implementation_effort
      - blast_radius_penalty

minimum_entry_criteria:
  stable_process: true
  named_owner: true
  test_environment: true
  rollback_defined: true
```

Production Checklist

- A baseline exists for time, failure rate, and operator effort.
- The process is stable enough to automate.
- Benefits and risk are scored consistently across proposals.
- Adoption, support, and retirement costs are included.
- Success metrics are reviewed after production rollout.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Many automations, little impact	Projects selected by enthusiasm rather than evidence	Rebuild portfolio from incidents, toil, lead time, and control gaps	Baseline dashboard, scoring worksheet
Savings cannot be verified	No pre-automation baseline	Instrument manual process before replacement and use conservative attribution	Before/after metrics, sampling method
Users keep manual process	Automation lacks required cases or is hard to access	Interview users, close functional gaps, and improve golden-path ergonomics	Adoption funnel, support tickets
Platform team overloaded	Every request becomes custom work	Create product tiers, reusable patterns, intake criteria, and self-service documentation	Backlog age, reuse rate, support load

Hands-on Lab 57: Build and Prioritize an Automation Portfolio

Prerequisites

- At least ten candidate tasks from a real or simulated operations team.
- Access to incident, ticket, or process timing data.
- A spreadsheet or issue tracker for scoring.

Implementation

1. Record each candidate process, owner, frequency, duration, failure rate, inputs, outputs, and affected systems.
2. Measure or estimate a conservative baseline and document confidence level.
3. Score value, risk reduction, feasibility, blast radius, and reversibility.
4. Reject candidates with unstable requirements or no accountable owner.
5. Select the top three opportunities and define a minimum viable automation product for each.
6. Create a quarterly review that compares predicted versus observed outcomes.

Validation Evidence

- Selected projects have documented baselines and owners.
- Scoring can be reproduced by another reviewer.
- At least one high-effort but low-value project is deliberately deferred.
- Success metrics include adoption and reliability, not only execution count.

Cleanup

- Remove sensitive ticket details from the shared portfolio.
- Archive assumptions so future reviewers understand score changes.

Interview Questions and Model Answers

1. What is the best source of automation ideas?

Operational evidence: recurring incidents, toil, lead time, failed changes, audit gaps, and capacity bottlenecks.

2. Why can a technically successful automation still fail?

Users may not trust it, it may omit real-world cases, support may be poor, or the manual path may remain easier.

3. How do you avoid exaggerated ROI?

Capture a baseline, use conservative attribution, include maintenance cost, and report confidence ranges.

4. When should a process not be automated?

When it is unstable, rarely repeated, lacks ownership, has ambiguous decisions, or cannot be tested and recovered safely.

Chapter Summary

- Automation portfolios should optimize outcomes, not script count.
- Baseline measurements precede credible ROI.
- Suitability and reversibility matter as much as potential savings.
- Adoption and lifecycle cost are part of the product.

Chapter 58: Multi-Account and Multi-Cloud Landing-Zone Automation Project

A landing-zone factory establishes secure, governed environments repeatedly across accounts, subscriptions, projects, or on-premises tenants. Terraform creates durable hierarchy, identity, networking, logging, guardrails, and shared services; Ansible configures approved runtime components and validates operational readiness.

Learning Objectives

- Design a reusable landing-zone factory with isolated state and delegated ownership.
- Implement organization-wide guardrails without blocking legitimate workload variation.
- Generate environment contracts for downstream configuration and application teams.
- Prove account creation, policy enforcement, logging, and teardown through automated tests.

Core Concepts

Factory inputs: A versioned request describes business owner, environment class, data classification, regions, connectivity, budget, retention, and approved exceptions.

Hierarchy and state: Separate organization foundations, shared services, network hubs, security services, and workload environments into independently owned states with explicit outputs.

Identity bootstrap: Use federated identities, least-privilege deployment roles, break-glass controls, and separate human from workload access.

Guardrails: Enforce mandatory logging, encryption, region restrictions, public-access controls, tagging, budget alerts, and service allowlists through provider-native policy and pipeline checks.

Configuration handoff: Ansible receives approved host or service endpoints through a reduced contract, then applies hardening, agents, certificates, and operational validation.

Lifecycle: Environment suspension, ownership transfer, exception expiry, evidence retention, and deletion are first-class factory workflows, not manual afterthoughts.

Reference Configuration or Command Pattern

```
# environment request contract
environment_id: payments-prod
owner_group: team-payments
data_classification: confidential
regions: [primary-region, recovery-region]
connectivity_profile: private-hub
logging_profile: regulated
budget_monthly: 25000
exceptions:
  - control: approved-service-list
    expires: 2026-09-30
  approver: risk-owner
```

Production Checklist

- Foundation and workload states are separated by ownership and blast radius.
- Deployment identity cannot alter unrelated environments.
- Policies are testable and exceptions expire automatically.
- Central logging and security telemetry are verified from a workload.

- Deletion preserves required evidence and removes residual access.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
New environment cannot deploy	Foundation output or delegated role is missing	Validate contract version, role trust, quotas, and provider registration	Request artifact, plan log, IAM trace
Policy blocks approved workload	Guardrail lacks scoped exception mechanism	Use time-bound exception with compensating controls and owner	Policy result, exception record
Logs absent centrally	Sink or destination permission incomplete	Test end-to-end log emission and destination authorization	Sample event, sink status, access log
Destroy leaves resources	Cross-state dependency or protection setting	Use retirement workflow, dependency inventory, and explicit evidence retention	Destroy plan, residual inventory

Hands-on Lab 58: Build a Two-Environment Landing-Zone Factory

Prerequisites

- Two disposable cloud accounts, subscriptions, projects, or simulated tenants.
- Terraform remote state with locking.
- Ansible access to at least one provisioned host or service endpoint.

Implementation

1. Define a request schema and validation rules for development and production classes.
2. Provision hierarchy, deployment roles, networking, logging, budgets, and mandatory labels with versioned modules.
3. Add policy tests that intentionally reject public storage, missing encryption, and unapproved regions.
4. Generate a reduced handoff artifact and configure a baseline host with Ansible.
5. Verify central logs, identity boundaries, network reachability, and budget alerts.
6. Run the retirement workflow for development and prove that production is unaffected.

Validation Evidence

- Development and production use separate state and deployment identities.
- Negative policy tests fail before apply.
- A centrally visible event proves telemetry delivery.
- Retirement removes the target environment without modifying shared foundations.

Cleanup

- Destroy disposable environments in dependency order.
- Revoke federation subjects and remove temporary exception records.

Interview Questions and Model Answers

1. Why split foundation and workload state?

They have different owners, privileges, change rates, and blast radius. Separate state prevents ordinary workload changes from modifying organization foundations.

2. What is a landing-zone contract?

A versioned request and output schema defining environment requirements, policies, identity, connectivity, telemetry, and downstream interfaces.

3. How should exceptions work?

They should be narrowly scoped, time-bound, approved by a risk owner, visible in evidence, and paired with compensating controls.

4. What validates a landing zone?

Negative guardrail tests plus end-to-end checks for identity, connectivity, logging, cost controls, and lifecycle operations.

Chapter Summary

- Landing zones are lifecycle factories, not one-time account scripts.
- State and identity boundaries contain blast radius.
- Guardrails require testable, expiring exceptions.
- Operational telemetry and retirement must be proven end to end.

Chapter 59: Kubernetes and Application-Platform Automation Project

Application platforms combine cloud infrastructure, Kubernetes, networking, identity, secrets, observability, policy, and developer workflows. Terraform should create stable platform primitives while Ansible or specialized operators configure nodes, supporting services, and external integrations without competing with Kubernetes reconciliation.

Learning Objectives

- Design clear ownership among Terraform, Ansible, package managers, and Kubernetes operators.
- Automate platform provisioning, add-ons, policy, and verification safely.
- Implement progressive upgrades with compatibility and rollback gates.
- Provide developers with a supported golden path and measurable platform SLOs.

Core Concepts

Ownership layers: Terraform owns clusters, network, IAM, registries, load-balancer prerequisites, and durable managed services. Kubernetes manifests or operators own in-cluster resources. Ansible handles host-level or external systems when an API-native owner is not appropriate.

Bootstrap boundary: Bootstrap should be minimal and repeatable: identity integration, admission policy, networking, storage classes, telemetry, secret integration, and GitOps controller.

Version compatibility: Pin provider, Terraform, cluster, add-on, collection, execution-environment, and API versions. Test supported upgrade sequences rather than jumping multiple layers simultaneously.

Policy and tenancy: Namespaces alone are not complete isolation. Combine identity, network policy, quotas, admission controls, secret boundaries, and workload-specific service accounts.

Verification: Test scheduling, DNS, storage, ingress, workload identity, metrics, logs, traces, policy denial, and node disruption.

Developer experience: Golden paths should expose simple, versioned interfaces while retaining escape hatches with documented ownership and support consequences.

Reference Configuration or Command Pattern

```
# ownership map
terraform:
  - cluster_api
  - node_pools
  - network_and_identity
kubernetes_gitops:
  - namespaces
  - policies
  - platform_addons
ansible:
  - external_bastion_hardening
  - legacy_load_balancer_integration
forbidden_overlap:
  - node_labels_owned_by_cluster_api
```

Production Checklist

- No resource is reconciled by two automation systems.

- Upgrade compatibility is documented and tested in a lower environment.
- Workload identity replaces shared static credentials.
- Platform failure and rollback signals are observable.
- Developer templates have owners, versions, and deprecation policy.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Add-ons revert after Ansible run	GitOps or operator owns the same resource	Remove overlapping owner and reconcile through authoritative controller	Object managed fields, job log, Git commit
Cluster ready but workloads fail	DNS, CNI, storage, or identity bootstrap incomplete	Run layered conformance checks before application promotion	Probe results, events, component health
Upgrade stalls	Unsupported version combination or disruption budget	Validate compatibility matrix and capacity before rollout	Version inventory, drain events
Developer path is bypassed	Template is restrictive or slow	Measure friction, add safe parameters, and document exception route	Adoption metrics, feedback record

Hands-on Lab 59: Automate a Secure Application Platform and Upgrade

Prerequisites

- A disposable Kubernetes cluster or local multi-node equivalent.
- Terraform and a GitOps-capable manifest workflow.
- Ansible access to an external support host or simulated legacy integration.

Implementation

1. Document resource ownership across Terraform, Kubernetes, GitOps, and Ansible.
2. Provision the cluster, identity, network, registry access, and logging prerequisites.
3. Bootstrap policy, telemetry, secret integration, and one golden-path application.
4. Execute negative tests for privilege, network isolation, and unapproved image source.
5. Perform a controlled node or add-on upgrade with canary verification and rollback threshold.
6. Publish platform SLOs, support boundaries, and upgrade evidence.

Validation Evidence

- Managed fields show one authoritative reconciler for each object.
- Golden-path application passes identity, network, storage, and telemetry tests.
- Policy rejects an intentionally noncompliant workload.
- Upgrade completes within disruption and error-budget limits.

Cleanup

- Delete disposable workloads and cluster resources.
- Revoke temporary registry and federation permissions.

Interview Questions and Model Answers

1. Why not manage all Kubernetes resources with Terraform?

Terraform can do so, but continuous in-cluster reconciliation, frequent application releases, and operator-managed resources often fit Kubernetes-native or GitOps controllers better.

2. What is the bootstrap boundary?

The smallest set of platform capabilities required before ordinary workload delivery can proceed safely and repeatedly.

3. How do you prevent automation conflict?

Publish an ownership map, inspect managed fields and state, and reject changes to resources owned by another reconciler.

4. What is the most valuable platform test?

An end-to-end workload test that proves identity, networking, storage, policy, telemetry, and recovery rather than only cluster API availability.

Chapter Summary

- Application platforms require explicit reconciliation ownership.
- Bootstrap should be minimal, pinned, and repeatable.
- Compatibility and disruption gates govern upgrades.
- Developer experience is an operational requirement.

Chapter 60: Data, Middleware, and Stateful-Service Automation Project

Stateful automation must protect data integrity while coordinating infrastructure, configuration, schema, backup, replication, and application dependencies. The safest design separates resource provisioning from data-plane change and requires restore evidence before claiming recoverability.

Learning Objectives

- Automate stateful services without conflating infrastructure and data ownership.
- Design backup, restore, migration, and credential rotation workflows.
- Use maintenance gates and application-aware verification.
- Prevent destructive replacement and accidental data loss.

Core Concepts

Lifecycle separation: Terraform provisions managed databases, storage, queues, caches, DNS, and identities. Database migration tools or application pipelines own schema and data change. Ansible configures self-managed services and client integration.

Destruction controls: Use lifecycle protections, policy gates, independent backups, deletion protection, and explicit break-glass review. Protection is not a substitute for tested restore.

Secret rotation: Applications must tolerate overlapping credentials or certificates during rotation. Automate issuance, distribution, verification, cutover, and revocation as separate stages.

Maintenance orchestration: Coordinate quiesce, replica health, backup, change, smoke test, traffic restoration, and monitoring. Make each stage resumable.

Data verification: Validate record counts, checksums, replication lag, query correctness, queue depth, cache behavior, and application transactions.

Recovery evidence: Record backup identifier, consistency point, restore target, elapsed time, data-loss window, validation results, and cleanup.

Reference Configuration or Command Pattern

```
# stateful change gate
preconditions:
  backup_age_minutes_max: 30
  replica_lag_seconds_max: 5
  active_incident: false
  restore_test_age_days_max: 30
change:
  serial: true
  rollback: restore_or_forward_fix
postconditions:
  synthetic_transaction: pass
  error_rate_percent_max: 1
```

Production Checklist

- Infrastructure, schema, and data ownership are separate.
- Deletion and replacement require explicit high-risk approval.
- Backup restore is tested against stated RTO and RPO.
- Credential rotation supports overlap and rollback.

- Application-level transactions validate the final state.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Terraform proposes database replacement	Immutable argument changed or address refactored incorrectly	Stop apply, inspect provider schema and state address, use safe migration or moved block	Saved plan, schema diff, state backup
Configuration succeeds but app fails	Client trust, credentials, DNS, or schema incompatible	Run application transaction and dependency checks before promotion	Connection trace, app logs, schema version
Backup exists but restore fails	Backup incomplete, permissions missing, or procedure untested	Schedule isolated restore drills and validate data	Backup metadata, restore log, checksum
Rotation causes outage	Old secret revoked before all clients updated	Use dual credential window and inventory of consumers	Rotation timeline, client versions

Hands-on Lab 60: Automate a Stateful Service Change and Restore Drill

Prerequisites

- A disposable database, message broker, or stateful container service.
- Terraform provisioning code and Ansible configuration role.
- A sample application transaction and backup mechanism.

Implementation

1. Define ownership for infrastructure, configuration, schema, data, backups, and application release.
2. Provision the service with encryption, deletion protection, monitoring, and a backup policy.
3. Load sample data and capture a consistency baseline.
4. Rotate a credential using overlap, verify consumers, then revoke the old credential.
5. Introduce a controlled failure and restore to an isolated target.
6. Measure RTO, observed RPO, and application transaction correctness.

Validation Evidence

- No destructive replacement occurs during normal change.
- Credential rotation produces no failed transactions.
- Restored data matches the expected consistency point.
- Measured RTO and RPO are recorded against objectives.

Cleanup

- Destroy isolated restore targets after evidence capture.
- Revoke temporary credentials and remove sample data.

Interview Questions and Model Answers

1. Why separate schema migration from Terraform?

Schema and data changes have application-aware ordering, compatibility, and rollback semantics that differ from infrastructure resource lifecycle.

2. What is the difference between backup and recoverability?

A backup is an artifact; recoverability is proven by restoring it within objectives and validating data and application behavior.

3. How should secrets be rotated?

Issue new credentials, distribute them, verify all consumers, cut over, observe, and only then revoke the old credentials.

4. What should block a stateful change?

Stale or failed backups, excessive replication lag, active incidents, missing recovery capacity, or an untested destructive plan.

Chapter Summary

- Stateful automation separates infrastructure, configuration, schema, and data ownership.
- Destructive changes require strong policy and recovery evidence.
- Secret rotation is a staged compatibility exercise.
- Restore drills prove recoverability.

Chapter 61: Brownfield Discovery, Import, Refactoring, and Standardization

Most enterprises begin with infrastructure that already exists, contains undocumented dependencies, and was changed by multiple tools and humans. Brownfield adoption is an evidence-driven migration: discover, classify, model, import, reconcile, refactor, and only then enforce standard automation.

Learning Objectives

- Inventory existing resources and configuration without assuming ownership.
- Choose import, rebuild, retain-manual, or retire strategies.
- Refactor Terraform addresses and Ansible content without destructive change.
- Establish a controlled cutover to authoritative automation.

Core Concepts

Discovery: Combine provider inventory, configuration databases, tags, billing, network flows, DNS, certificates, backups, controller history, and stakeholder interviews.

Classification: Classify each object by owner, criticality, data state, replacement tolerance, drift, dependencies, and current automation authority.

Import strategy: Use configuration-driven import or CLI import only after the destination address and provider configuration are stable. Import records state binding; it does not automatically create correct design.

Reconciliation: The first plan is an investigation. Adjust configuration to represent intentional current behavior, then separately propose standardization changes.

Refactoring: Use moved blocks for durable Terraform address changes and deprecation-compatible role or collection interfaces for Ansible content. Preserve consumer migration paths.

Cutover: Freeze conflicting manual or legacy automation paths, capture backup, perform reviewed reconciliation, validate, and monitor closely before declaring the new owner authoritative.

Reference Configuration or Command Pattern

```
# brownfield decision record
resource_id: legacy-db-01
current_owner: operations-manual
strategy: import_then_standardize
replacement_tolerance: none
dependencies: [payments-api, backup-vault, private-dns]
cutover_controls:
  - state_backup
  - change_freeze
  - no_destroy_plan
  - app_transaction_test
```

Production Checklist

- Discovery uses multiple evidence sources.
- Every resource has a migration strategy and accountable owner.
- The first post-import plan is reviewed for unintended change.
- Address refactoring uses declarative migration where supported.
- Legacy automation is disabled only after successful cutover.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Import succeeds but plan changes everything	Configuration does not match current remote settings	Treat plan as discovery and model intentional attributes before apply	Import log, plan JSON, configuration diff
Resource will be destroyed after module move	Address change not recorded	Add and test moved block or controlled state move with backup	State backup, move plan
Ansible changes unexpected hosts	Legacy inventory and new dynamic inventory overlap	Freeze old path and prove target set before execution	Inventory graph, controller schedule
No owner approves cutover	Ownership or business criticality unknown	Escalate through service catalog and do not assume authority	Discovery record, approval trail

Hands-on Lab 61: Adopt a Brownfield Environment Without Replacement

Prerequisites

- A disposable manually created environment with at least five resources.
- Terraform provider access and Ansible connectivity.
- A documented application or service verification test.

Implementation

1. Discover resources, dependencies, owners, tags, access paths, and current configuration.
2. Classify each resource as import, rebuild, retain-manual, or retire.
3. Write Terraform configuration and import selected resources into isolated remote state.
4. Review the first plan and eliminate unintended destructive or reset changes.
5. Refactor at least one address using a moved block and convert one manual configuration into an idempotent Ansible role.
6. Freeze the old path, run cutover, verify application behavior, and record the authoritative owner.

Validation Evidence

- The final Terraform plan has no unintended destroy or replacement.
- A second Ansible run is idempotent.
- Dependencies and target hosts are explicitly verified.
- The old automation path is disabled and rollback remains documented.

Cleanup

- Destroy only disposable lab resources after exporting sanitized evidence.
- Remove temporary state copies containing sensitive values.

Interview Questions and Model Answers

1. What does Terraform import do?

It creates a binding between a configuration address and an existing remote object; it does not guarantee the configuration matches or that the design is correct.

2. Why is the first plan after import dangerous?

It may propose changing defaults, immutable fields, tags, networking, or other attributes that were not modeled accurately.

3. When should an object be rebuilt instead of imported?

When replacement is safe and cheaper, the current design is unsupportable, or clean recreation provides a controlled path to the desired standard.

4. How do you prevent brownfield dual ownership?

Freeze manual and legacy automation during cutover, publish the new authority, and monitor for out-of-band changes.

Chapter Summary

- Brownfield adoption starts with discovery and classification.
- Import is state binding, not design conversion.
- Reconciliation precedes standardization.
- Cutover must remove conflicting authority.

Chapter 62: Disaster Recovery, Rebuild Automation, and Continuity Testing

Infrastructure automation can accelerate recovery only when state, source, artifacts, identity, dependencies, and data backups remain available during the disaster. A credible recovery design distinguishes restore, rebuild, failover, and reconciliation and tests them under realistic constraints.

Learning Objectives

- Design recovery for automation control planes and managed environments.
- Protect and restore Terraform state, controller configuration, inventories, and artifact registries.
- Create deterministic rebuild workflows with dependency ordering.
- Measure recovery objectives through game days.

Core Concepts

Recovery modes: Restore returns a control plane or data set from backup; rebuild recreates it from code; failover activates an alternate environment; reconciliation brings the recovered system back under authoritative automation.

State protection: Use remote versioning, locking, access controls, backups, and tested state pull/push procedures. Manual state modification is a last-resort recovery action.

Controller continuity: Back up controller database and configuration, credential references, execution-environment images, project sources, inventories, schedules, and signing or trust configuration.

Dependency ordering: Identity, DNS, networking, registries, state services, secrets, and artifact stores often precede workload restoration. Record bootstrap dependencies that cannot be created by the failed platform itself.

Immutable evidence: Store source, module and collection versions, lock files, plans, execution images, SBOMs, policy results, and verification artifacts in durable systems with independent access.

Game-day design: Define scenario, assumptions, safety guardrails, start state, injects, roles, success criteria, stop conditions, and evidence before beginning.

Reference Configuration or Command Pattern

```
# recovery runbook stages
1. declare incident and freeze automation
2. verify identity and evidence access
3. restore or rebuild state/control plane
4. validate serials, locks, and workspace bindings
5. plan in read-only or refresh mode
6. restore dependencies and workloads
7. run independent verification
8. resume controlled automation and reconcile drift
```

Production Checklist

- State and controller backups are encrypted, versioned, and restore-tested.
- Bootstrap dependencies are documented outside the primary platform.
- Recovery uses pinned source and artifacts.
- RTO and RPO are measured, not assumed.
- Post-recovery reconciliation avoids accidental replacement.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
State backend unavailable	Regional outage or identity dependency failed	Use documented alternate access or restore path; freeze all applies	Incident record, backend status, state backup
Recovered state rejected	Serial or lineage conflict	Compare metadata and use state push only after peer review and backup	State versions, approval, push log
Controller restored but jobs fail	Execution images, credentials, or projects missing	Restore dependencies and validate one low-risk job first	Controller inventory, job events
Rebuild creates duplicate resources	Existing remote objects not reconciled	Perform discovery and import/rebind before apply	Refresh plan, import record

Hands-on Lab 62: Execute an Automation Control-Plane Recovery Game Day

Prerequisites

- A disposable Terraform remote backend and Ansible controller or AWX lab.
- Versioned source and recoverable execution artifacts.
- A small managed environment with verification tests.

Implementation

1. Define RTO, RPO, safety guardrails, roles, stop conditions, and recovery mode.
2. Capture state backup, controller export or backup, artifact digests, and dependency inventory.
3. Simulate loss of the primary state or controller service and freeze new changes.
4. Restore or rebuild the control plane using documented bootstrap credentials.
5. Run non-destructive Terraform inspection and a low-risk Ansible verification job.
6. Resume controlled operation, reconcile drift, and publish measured recovery results.

Validation Evidence

- Recovery uses only documented sources and artifacts.
- No duplicate or unintended replacement is created.
- State lineage, locks, inventories, and credentials are verified.
- Measured RTO and RPO are compared with objectives and gaps assigned.

Cleanup

- Remove simulated outage controls and temporary recovery credentials.
- Destroy disposable recovery infrastructure after evidence retention.

Interview Questions and Model Answers

1. Why is IaC not automatically disaster recovery?

Code alone does not preserve state, data, identity, artifacts, DNS, dependencies, or tested operational procedure.

2. When is state push appropriate?

Only during controlled recovery after validating lineage and serial, taking a backup, freezing concurrent work, and receiving peer approval.

3. What is a bootstrap dependency?

A service required to rebuild the automation platform that the failed platform cannot recreate until that dependency already exists.

4. What makes a game day credible?

Realistic constraints, measurable objectives, explicit stop conditions, independent observation, and tracked remediation afterward.

Chapter Summary

- Recovery requires source, state, identity, artifacts, dependencies, and data.
- Restore, rebuild, failover, and reconciliation are distinct actions.
- Manual state operations are controlled recovery tools.
- Game days convert assumptions into evidence.

Chapter 63: Advanced Terraform Troubleshooting and State Incident Response

Terraform incidents require separation of configuration, provider, state, backend, identity, API, and remote-resource evidence. The safest responder first freezes concurrent change, preserves state and plans, classifies the failure, and chooses the least invasive recovery action.

Learning Objectives

- Apply a structured Terraform incident-triage workflow.
- Diagnose locks, partial applies, drift, provider failures, and address errors.
- Use state commands safely and understand their blast radius.
- Recover without hiding unresolved infrastructure or data risk.

Core Concepts

Evidence first: Capture Terraform version, providers and lock file, configuration commit, environment variables, backend, workspace, plan, logs, state serial and lineage, API errors, and recent out-of-band changes.

Plan taxonomy: Distinguish configuration change, refresh drift, replacement, unknown value propagation, address move, import, dependency ordering, and provider-computed differences.

Partial apply: Terraform can update state for completed actions before a later action fails. The next plan is the source of truth; do not assume a full rollback occurred.

Lock incidents: Confirm no valid process owns the lock before force-unlock. Removing a legitimate lock can permit concurrent state writes.

State operations: Prefer declarative moved, removed, and import blocks when possible. Before manual state mv, rm, or push, freeze changes and pull a backup.

Provider diagnosis: Reproduce with pinned versions, inspect provider logs carefully for secrets, validate credentials and API quotas, and reduce the case without changing production state.

Reference Configuration or Command Pattern

```
# incident-safe Terraform triage
terraform version
terraform providers
terraform validate
terraform state pull > incident-state-backup.json
terraform plan -refresh-only -out=refresh.tfplan
terraform show -json refresh.tfplan > refresh.json
# Do not apply, force-unlock, or push state until ownership and evidence are verified.
```

Production Checklist

- Concurrent runs are stopped before recovery.
- State is backed up and lineage recorded.
- Plan is reviewed in saved and JSON form where appropriate.
- The least invasive declarative recovery is preferred.
- Independent service verification follows any corrective apply.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
State lock remains after crash	Process terminated before release	Verify no active owner, record lock ID, then controlled force-unlock	Backend logs, process check, approval
Apply failed halfway	API error, quota, dependency, or provider defect	Inspect current state and next plan; repair cause and resume safely	Apply log, state serial, next plan
Unexpected replacement	Immutable field, address change, key instability, or provider behavior	Stop; inspect replace paths, moved blocks, for_each keys, and schema	Plan JSON, provider version, state backup
State and reality diverge	Out-of-band change or failed import/recovery	Use refresh-only evidence and decide adopt, revert, or replace	Drift report, change audit, decision record

Hands-on Lab 63: Resolve a Terraform State and Partial-Apply Incident

Prerequisites

- A disposable Terraform environment with remote state and locking.
- Permission to inspect backend version history.
- A controlled way to cause one resource action to fail.

Implementation

1. Create a baseline apply and record state serial, lineage, provider versions, and service checks.
2. Trigger a controlled partial apply by denying or exhausting one later API action.
3. Freeze runs and capture logs, state backup, failed plan, and remote-resource inventory.
4. Correct the root cause and generate a new plan without assuming rollback.
5. Simulate a stale lock and execute the verification and approval process before force-unlock.
6. Run recovery apply, independent verification, and a blameless incident report.

Validation Evidence

- The recovery plan reflects completed and failed actions accurately.
- No concurrent writer exists when the lock is removed.
- State serial advances predictably and remote resources match intended configuration.
- Incident report identifies prevention and detection improvements.

Cleanup

- Destroy disposable resources with a reviewed plan.
- Securely remove local state and verbose logs.

Interview Questions and Model Answers

1. What should you do first during a Terraform state incident?

Stop concurrent changes, preserve evidence, identify the correct workspace and backend, and pull a state backup.

2. Does a failed apply roll everything back?

No. Completed actions may be recorded in state; inspect the next plan and real infrastructure.

3. When is force-unlock unsafe?

When a valid run still owns the lock or when ownership has not been verified.

4. Why prefer moved and removed blocks?

They make state intent reviewable and repeatable through normal plan and apply workflows instead of one-time operator commands.

Chapter Summary

- Terraform incidents require evidence across code, provider, state, backend, API, and resources.
- Partial apply is normal and not an automatic rollback.
- Lock and state commands require change freeze and backup.
- The next reviewed plan guides recovery.

Chapter 64: Advanced Ansible Troubleshooting and Controller Incident Response

Ansible failures can originate in inventory, variable precedence, connection plugins, privilege escalation, execution environments, content versions, target state, controller scheduling, or the remote module itself. Effective responders narrow the failure by layer and reproduce with the same execution context.

Learning Objectives

- Diagnose Ansible failures from controller to target.
- Use verbosity, inventory inspection, check mode, step mode, and debugger safely.
- Recover failed rolling changes without broad reruns.
- Troubleshoot automation controller jobs, credentials, projects, and execution nodes.

Core Concepts

Context capture: Record ansible-core and collection versions, execution-environment digest, inventory source revision, variable sources, limit, tags, forks, strategy, controller template, credential IDs, and target facts.

Layered diagnosis: Test inventory resolution, name and group membership, connection, interpreter, privilege escalation, module execution, handler behavior, service health, and application verification separately.

Variable evidence: Use inventory graph and host output, explicit assertions, and carefully redacted debug statements. Avoid printing secrets or entire hostvars.

Failure semantics: Understand any_errors_fatal, max_fail_percentage, serial batches, ignore_errors, failed_when, changed_when, block/rescue/always, and handler notification behavior.

Controller incidents: Project sync, inventory sync, credential injection, capacity, execution node health, image pull, receptor connectivity, and job isolation can fail before the playbook starts.

Safe resume: Use idempotent tasks, checkpoints, tags, start-at-task with caution, and host limits. Dynamic includes and hidden side effects make arbitrary resume unsafe.

Reference Configuration or Command Pattern

```
# layered Ansible triage
ansible-inventory -i inventory.yml --graph
ansible-inventory -i inventory.yml --host target01
ansible target01 -i inventory.yml -m ansible.builtin.ping -vvv
ansible-playbook site.yml -i inventory.yml --syntax-check
ansible-playbook site.yml -i inventory.yml --check --diff --limit target01
# Reproduce inside the same execution-environment image used by the controller.
```

Production Checklist

- Controller and local reproduction use the same content and execution image.
- Target set and variable sources are proven before change.
- Verbose logs are sanitized for credentials and sensitive values.
- Resume scope is limited to verified hosts and stages.
- Application behavior is validated after convergence.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Works locally, fails in controller	Different execution image, inventory, credentials, path, or network	Reproduce in controller image and compare job artifacts	EE digest, inventory revision, credential IDs
Host unreachable	DNS, route, key, user, host key, connection plugin, or timeout	Test connection layer separately with high verbosity	Connection trace, target logs
Task reports changed every run	Non-idempotent command or inaccurate changed_when	Use desired-state module or explicit detection	First/second run diff
Rolling update stops inconsistently	Failure thresholds, handlers, or batch variables misunderstood	Model serial and failure semantics; validate each batch	Batch timeline, host results

Hands-on Lab 64: Investigate a Controller and Rolling-Deployment Failure

Prerequisites

- An AWX or Automation Controller lab, or a reproducible containerized execution environment.
- At least three disposable targets.
- A rolling playbook with health verification.

Implementation

1. Capture the controller template, project revision, inventory source, credential references, execution image digest, and launch parameters.
2. Introduce an inventory grouping error and prove it with inventory graph and host inspection.
3. Introduce a connection or privilege failure on one host and isolate the failing layer.
4. Run a serial deployment with a controlled application-health failure and observe batch behavior.
5. Correct the cause and resume only the verified scope using idempotent tasks and limits.
6. Publish a sanitized incident timeline and prevention controls.

Validation Evidence

- Local reproduction uses the same execution image and content versions.
- The intended target set is proven before rerun.
- Healthy hosts are not needlessly changed during recovery.
- A second full run is idempotent and service checks pass.

Cleanup

- Remove test credentials, failed schedules, and temporary inventory sources.
- Restore target services and destroy disposable hosts.

Interview Questions and Model Answers

1. Why does an Ansible job work locally but fail in controller?

The controller may use a different execution image, collection version, working directory, inventory revision, credential injection method, or network path.

2. What is the safest way to debug variables?

Inspect inventory and targeted assertions, redact output, and avoid dumping complete hostvars or secrets.

3. When is start-at-task risky?

When previous tasks established state, dynamic includes hide boundaries, handlers were skipped, or the playbook has non-idempotent side effects.

4. How do you recover a failed rolling deployment?

Stop promotion, identify the affected batch, restore service health, correct the cause, and resume only validated hosts before performing a full convergence check.

Chapter Summary

- Ansible troubleshooting is layered from controller through inventory and connection to module and service.
- Execution context must match production.
- Resume depends on idempotency and explicit checkpoints.
- Controller metadata is essential incident evidence.

Chapter 65: Integrated Pipeline, Identity, and Supply-Chain Incident Response

An automation pipeline can be technically correct and still dangerous if its identity, source, artifact, approval, or evidence chain is compromised. Incident response must determine what code ran, which identity authorized it, what artifacts were consumed, which targets changed, and whether trust must be re-established from a clean root.

Learning Objectives

- Investigate compromised pipeline credentials, source, and artifacts.
- Correlate Terraform runs, Ansible jobs, commits, identities, and target changes.
- Contain automation without destroying forensic evidence.
- Rebuild trust through keyless identity, verified artifacts, and controlled re-promotion.

Core Concepts

Chain of custody: Link source commit, pull request, reviewer, build identity, dependency locks, module and collection digests, execution image, plan digest, approval, apply identity, controller job, target set, and verification result.

Containment: Disable triggers and credentials, pause schedules, protect state, preserve logs and artifacts, and restrict runner networks. Avoid deleting compromised resources before evidence capture.

Identity analysis: Determine token issuer, subject, audience, claims, lifetime, role assumption, credential scope, and every use during the exposure window.

Artifact trust: Verify registry immutability, signatures, provenance, SBOM, dependency sources, lock files, and whether build or execution images can be reproduced.

Target reconciliation: Inventory all potentially changed resources and hosts, compare with authoritative configuration, and decide revert, rebuild, rotate, or accept through a reviewed incident change.

Trust restoration: Rotate credentials, rebuild runners and execution environments, revoke artifacts, revalidate source history, and promote from a known-clean commit with enhanced monitoring.

Reference Configuration or Command Pattern

```
# incident correlation envelope
incident_id: INC-2026-065
source_commit: abc123
build_identity: oidc:repo/platform:ref/main
terraform_run_id: run-...
plan_digest: sha256:...
ansible_job_id: 8452
execution_environment: registry/ee@sha256:...
target_contract_digest: sha256:...
verification_report: verify-065.json
```

Production Checklist

- Triggers are paused without destroying logs or state.
- Exposed identities and artifact digests are enumerated.
- Affected targets are discovered independently of pipeline records.
- Trust roots and runners are rebuilt before resuming.
- The first clean promotion receives enhanced review and monitoring.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Unknown changes appear	Compromised token or untracked manual action	Correlate cloud audit, controller, repository, and runner logs	Identity claims, target audit, job records
Artifact signature valid but malicious	Trusted builder or source was compromised	Verify provenance inputs and rebuild from known-clean source	Provenance, source history, builder attestation
Cannot identify target scope	Dynamic inventory or outputs were not retained	Query independent asset sources and controller events	Asset export, inventory snapshots
Pipeline resumes too early	Pressure to restore delivery before trust is rebuilt	Use explicit containment exit criteria and incident commander approval	Exit checklist, approval record

Hands-on Lab 65: Run a Pipeline Credential Compromise Tabletop and Re-Promotion

Prerequisites

- A sample CI pipeline with workload identity or short-lived credentials.
- Terraform and Ansible run metadata.
- A registry or local artifact store with digests.

Implementation

1. Define an incident scenario in which a runner token or repository workflow is compromised.
2. Pause triggers and preserve source, plans, state metadata, controller events, audit logs, and artifact manifests.
3. Trace the identity subject, permissions, issuance, and all target actions in the exposure window.
4. Build an independent affected-resource and host inventory.
5. Rebuild the runner or execution environment, revoke exposed trust, and create new verified artifacts.
6. Re-promote a known-clean commit with peer approval and enhanced post-deployment verification.

Validation Evidence

- Every changed target maps to an identity and evidence source or is explicitly unresolved.
- Compromised credentials and artifacts cannot be reused.
- The clean promotion uses rebuilt execution infrastructure.
- Containment exit criteria and residual risks are documented.

Cleanup

- Revoke tabletop credentials and delete simulated malicious artifacts.
- Retain only sanitized incident evidence.

Interview Questions and Model Answers

1. What is the first goal in a pipeline compromise?

Contain further automation while preserving evidence and protecting state and logs.

2. Why is a valid signature insufficient?

If the trusted builder or source was compromised, it can sign a malicious artifact; provenance inputs and trust roots must also be verified.

3. How do you find affected targets?

Correlate pipeline and controller records with independent cloud, host, network, and asset inventories.

4. When can delivery resume?

After credentials and trust roots are remediated, execution infrastructure is clean, affected targets are reconciled, and explicit exit criteria are approved.

Chapter Summary

- Automation incidents require end-to-end chain-of-custody evidence.
- Containment preserves state and forensics.
- Identity and artifact trust must be rebuilt, not merely rotated superficially.
- Independent target discovery validates pipeline records.

Chapter 66: Performance, Scale, Reliability, and Capacity Engineering

At enterprise scale, automation becomes a distributed system constrained by provider APIs, state locks, controller queues, network latency, target capacity, artifact distribution, and human approvals. Performance work must preserve correctness and blast-radius controls rather than simply increasing concurrency.

Learning Objectives

- Measure Terraform and Ansible performance by stage.
- Design state, runner, controller, and inventory topology for scale.
- Tune concurrency without exceeding API or service safety limits.
- Define capacity, backpressure, and degradation policies.

Core Concepts

Stage metrics: Measure queue wait, initialization, provider download, refresh, plan, policy, approval, apply, inventory sync, job startup, connection setup, task execution, verification, and evidence upload.

Terraform scale: Reduce oversized states, unstable dependencies, repeated data queries, and unnecessary refresh cost. Cache providers safely and separate independent failure domains.

Ansible scale: Tune forks, serial, throttle, strategy, pipelining, fact gathering, inventory caching, execution-node capacity, and artifact size based on target and network behavior.

Backpressure: Use queues, quotas, rate limits, admission policies, and retry with jitter. Unbounded retries can amplify provider or controller outages.

Safety envelope: Concurrency limits should consider target redundancy, disruption budget, API quotas, shared dependencies, observability, and operator ability to respond.

Capacity planning: Forecast environment count, resources per state, runs per hour, average and percentile duration, controller slots, artifact storage, log volume, and recovery headroom.

Reference Configuration or Command Pattern

```
# capacity worksheet
peak_runs_per_hour: 120
p95_plan_minutes: 8
p95_apply_minutes: 20
controller_concurrent_jobs: 40
max_hosts_per_batch: 25
api_rate_limit_headroom_percent: 30
recovery_reserved_capacity_percent: 20
degradation_policy: queue_noncritical_and_block_destructive
```

Production Checklist

- Performance is measured by stage and percentile.
- Concurrency changes are load-tested in disposable environments.
- Rate limiting and bounded retry prevent outage amplification.
- Critical recovery capacity is reserved.
- Optimization does not remove verification or approval controls.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Plans become progressively slower	Oversized state, expensive refresh, or provider API latency	Profile stage timing and decompose by ownership and lifecycle	Run timing, state size, API metrics
Controller queue grows	Insufficient execution capacity or long blocked jobs	Add capacity, classify workloads, and enforce timeouts	Queue depth, job duration, node capacity
High forks cause failures	Target or network saturation	Use serial/throttle and measure service health	Connection errors, target load
Retries worsen outage	Synchronized unbounded retry storm	Apply exponential backoff, jitter, circuit breaking, and queue limits	Retry counts, API rate metrics

Hands-on Lab 66: Benchmark and Safely Scale an Automation Workload

Prerequisites

- A Terraform project with enough resources to produce measurable stages.
- An Ansible inventory of at least ten disposable targets or containers.
- Metrics collection for runner and target resource usage.

Implementation

1. Capture baseline stage timings, p50/p95 duration, API calls, queue time, CPU, memory, network, and failure rate.
2. Change one Terraform factor such as state decomposition or data-query reuse and compare results.
3. Test Ansible forks, serial, pipelining, and fact gathering one variable at a time.
4. Introduce provider or target rate limiting and observe retry behavior.
5. Define safe concurrency and a degradation policy for noncritical and destructive work.
6. Publish a capacity forecast with recovery headroom and trigger thresholds.

Validation Evidence

- Results include repeated trials and percentiles rather than one run.
- Optimization does not increase failed change or service disruption.
- Rate-limit tests show bounded retry and backpressure.
- Capacity forecast identifies a scale limit and expansion action.

Cleanup

- Destroy benchmark resources and clear generated artifacts.
- Restore conservative production-like concurrency defaults.

Interview Questions and Model Answers

1. Why not maximize Terraform parallelism or Ansible forks?

Provider APIs, target services, networks, state locks, and operator response capacity impose safety limits; excess concurrency can create correlated failure.

2. What is the first performance metric to collect?

End-to-end stage timing with queue wait and percentiles, so the actual bottleneck is visible.

3. How does state decomposition improve scale?

It reduces refresh and lock scope and enables independent ownership, but creates interface and orchestration costs.

4. What is backpressure?

A controlled mechanism that slows or rejects incoming work when downstream capacity is saturated, preventing cascading failure.

Chapter Summary

- Automation scale is constrained by multiple distributed-system bottlenecks.
- Measure stages before tuning.
- Concurrency must remain inside a safety envelope.
- Backpressure and recovery headroom are production requirements.

Chapter 67: Automation Platform Product Management and Governance

A mature automation platform balances centralized guardrails with federated delivery. Governance should make safe behavior easier, expose ownership and evidence, and provide a controlled path for legitimate exceptions rather than forcing teams into unobservable bypasses.

Learning Objectives

- Design a federated automation operating model.
- Create golden paths, service catalog entries, and support commitments.
- Govern modules, collections, policies, versions, and exceptions.
- Measure platform health, adoption, risk, and customer outcomes.

Core Concepts

Federated model: A central platform team owns core services, standards, identity, registries, and golden paths; domain teams own workload-specific modules and roles within compatibility and security contracts.

Product interfaces: Publish supported inputs, outputs, examples, SLOs, ownership, cost model, version policy, migration guides, and support channels.

Quality tiers: Distinguish experimental, community, supported, critical, and deprecated content. Apply stronger testing and response expectations to higher-impact tiers.

Governance mechanisms: Use code ownership, protected branches, signed artifacts, policy gates, registry permissions, dependency allowlists, lifecycle metadata, and automated evidence.

Exception system: Exceptions need scope, rationale, owner, compensating controls, expiry, review, and automatic re-evaluation.

Metrics: Track adoption, lead time, failed-change rate, drift age, exception age, security findings, support load, reuse, upgrade lag, and consumer satisfaction.

Reference Configuration or Command Pattern

```
# automation product metadata
name: network-baseline
owner: platform-networking
tier: critical
interfaces: v3
supported_versions:
  terraform: '>=1.12,<2.0'
  ansible_core: '>=2.17,<2.20'
slo:
  support_response_minutes: 30
  release_success_percent: 99.5
  deprecation_notice_days: 180
```

Production Checklist

- Every automation product has an owner and service catalog entry.
- Version and deprecation policies are enforced.
- Critical content has stronger qualification and support.
- Exceptions expire and are visible to risk owners.
- Metrics include consumer outcomes and operational risk.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Teams fork critical modules	Interfaces or release cadence do not meet needs	Add extension points, improve roadmap transparency, and support migration	Fork inventory, consumer interviews
Old versions never retire	No ownership or deprecation enforcement	Publish deadlines, automated warnings, and upgrade assistance	Version adoption dashboard
Policy exceptions accumulate	Expiry and review not automated	Require owner, end date, compensating control, and alerting	Exception age report
Platform metrics look good but users complain	Only execution metrics are tracked	Measure lead time, support burden, satisfaction, and bypass rate	Survey, ticket trends, adoption funnel

Hands-on Lab 67: Design an Automation Platform Product and Governance Model

Prerequisites

- A sample module, role, collection, or pipeline used by multiple teams.
- A service catalog or documentation repository.
- A policy or approval system for exceptions.

Implementation

1. Define consumers, jobs to be done, ownership, support model, and success metrics.
2. Publish a versioned interface, examples, compatibility range, and deprecation policy.
3. Classify the product tier and define required tests, review, signing, and response time.
4. Create an exception workflow with expiry and compensating controls.
5. Build a dashboard for adoption, upgrade lag, failures, drift, exceptions, and support load.
6. Run a consumer review and incorporate one measured usability improvement.

Validation Evidence

- Consumers can identify the supported path and owner.
- A deprecated version produces an actionable warning or block.
- An expired exception is detected automatically.
- Dashboard combines technical reliability with consumer outcomes.

Cleanup

- Close temporary exceptions and test catalog entries.
- Remove synthetic usage data from production dashboards.

Interview Questions and Model Answers

1. What should a central platform team own?

Shared control planes, identity integration, policy, registries, golden paths, core modules, observability, and platform support - not every domain-specific resource.

2. Why use quality tiers?

Not all content has equal blast radius; critical products require stronger testing, support, and change control than experiments.

3. What makes an exception safe?

Narrow scope, named owner, rationale, compensating controls, expiry, visibility, and periodic reassessment.

4. Which metric reveals poor platform usability?

Bypass rate, time-to-first-success, support burden, and consumer satisfaction often reveal friction that success-count dashboards miss.

Chapter Summary

- Governance should enable safe federation.
- Automation content is a versioned supported product.
- Quality tiers align controls with blast radius.
- Outcome and adoption metrics complement technical reliability.

Chapter 68: Terraform and Ansible Certification Mastery

Certifications are most valuable when they organize study around practical capability. As of the 2026 edition, HashiCorp lists Terraform Associate (004) and Terraform Authoring and Operations Professional for infrastructure automation, while Red Hat maintains performance-based Ansible pathways such as RHCE and specialist exams. Always verify current official objectives before registration.

Learning Objectives

- Build a certification plan from current official exam objectives.
- Map each objective to a laboratory and evidence artifact.
- Prepare for knowledge-based and performance-based assessment styles.
- Use certification study to strengthen production skill rather than memorize isolated facts.

Core Concepts

Official source of truth: Use the current certification provider page, exam guide, version statement, policies, and sample format. Third-party blueprints can lag or contain inaccuracies.

Terraform Associate: Focus on Terraform workflow, language, state, modules, providers, HCP Terraform concepts, and safe lifecycle fundamentals. Current official prep materials identify Associate 004.

Terraform Professional: The Authoring and Operations Professional path emphasizes advanced module authoring and organizational operations; practical production experience is expected.

Red Hat performance exams: RHCE and Ansible specialist exams assess hands-on execution. Candidates must manage time, read requirements precisely, validate results, and recover from mistakes in a controlled environment.

Lab mapping: For every objective, create a build lab, a failure lab, a troubleshooting note, and a concise explanation. Mark weak areas through observed evidence, not confidence alone.

Exam hygiene: Practice under time limits, use official documentation rules, verify versions, avoid unauthorized exam content, and understand retake, expiration, and identification requirements.

Reference Configuration or Command Pattern

```
# objective evidence matrix
objective: manage-terraform-state
learn:
  - remote_backends_and_locking
  - import_moved_removed_blocks
prove:
  - state_recovery_lab
  - brownfield_import_lab
explain:
  - partial_apply_and_lineage
weakness_signal:
  - cannot_recover_without_unreviewed_state_push
```

Production Checklist

- Current official exam guide and tested product version are recorded.
- Every objective maps to at least one hands-on exercise.
- Practice includes failure recovery and time limits.
- Notes explain why a technique is safe, not only the command.
- No unauthorized dumps or recalled live exam content are used.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Practice scores high but labs fail	Recognition without operational skill	Convert each topic into build, break, recover, and explain exercises	Lab evidence, error journal
Study plan expands endlessly	Objectives not bounded by official guide	Create weighted objective matrix and defer unrelated depth	Current blueprint, weekly plan
Performance exam runs out of time	No triage or verification routine	Practice requirement parsing, checkpoints, and final validation	Timed attempts, missed-point analysis
Material conflicts on versions	Third-party content is stale	Use official versioned documentation and exam page	Source date, official guide

Hands-on Lab 68: Create a Certification Readiness and Evidence Plan

Prerequisites

- Current official certification and exam objective pages.
- A laboratory environment for Terraform and Ansible.
- A tracker for objectives, evidence, and weak areas.

Implementation

1. Select one Terraform and one Ansible-related certification target appropriate to your role.
2. Record official objectives, current version, format, prerequisites, and policy links.
3. Map every objective to a chapter, lab, command, troubleshooting scenario, and evidence artifact.
4. Run a timed baseline assessment without notes and classify gaps.
5. Complete build-break-recover-explain exercises for the five weakest areas.
6. Schedule two final simulations and a documentation refresh immediately before registration.

Validation Evidence

- All objectives trace to hands-on evidence.
- Weaknesses are based on observed errors or time, not self-rating alone.
- The plan references current official sources.
- Final simulation includes independent verification and cleanup.

Cleanup

- Delete temporary credentials and exam-lab resources.
- Retain a sanitized error journal for future revision.

Interview Questions and Model Answers

1. What is the best certification preparation method?

Map official objectives to repeated hands-on build, failure, recovery, and explanation exercises under realistic time constraints.

2. Why must the official guide be checked again before registration?

Exam names, versions, objectives, prerequisites, format, and policies can change.

3. How does performance-based preparation differ?

You must execute tasks accurately, manage time, validate outcomes, and recover from errors rather than only recognize correct statements.

4. What should you avoid?

Unauthorized exam dumps, memorized live questions, and unsupported shortcuts that undermine both ethics and real capability.

Chapter Summary

- Certification objectives change and official sources are authoritative.
- Hands-on evidence is stronger than passive study.
- Failure recovery and timed practice expose real gaps.
- Ethical preparation builds durable professional skill.

Chapter 69: Technical Interviews and Automation System Design

Senior automation interviews test reasoning under incomplete requirements. Strong candidates clarify outcomes, define ownership and failure domains, compare alternatives, expose security and recovery assumptions, and communicate a staged implementation with measurable acceptance criteria.

Learning Objectives

- Structure answers for Terraform, Ansible, platform, and incident questions.
- Design automation systems from requirements and constraints.
- Explain trade-offs, failure handling, and migration strategy.
- Present portfolio evidence clearly and honestly.

Core Concepts

Clarification: Ask about scale, environments, providers, data sensitivity, availability, team ownership, existing tools, change frequency, compliance, and recovery objectives before choosing architecture.

Answer structure: Use context, decision, implementation, controls, failure modes, verification, and trade-offs. For incidents use detect, contain, preserve evidence, diagnose, recover, validate, and prevent.

Terraform depth: Expect state design, modules, provider aliases, lifecycle, import, drift, policy, testing, secrets, HCP operations, and recovery.

Ansible depth: Expect inventory, precedence, idempotency, roles and collections, execution environments, controller architecture, rolling change, Windows or network differences, testing, and debugging.

System design: Describe source, identity, artifact, policy, state, inventory, execution, approval, observability, evidence, and target planes with trust and failure boundaries.

Portfolio storytelling: Use situation, constraints, actions, metrics, failures, corrections, and lessons. Do not exaggerate scale or claim team work as individual work.

Reference Configuration or Command Pattern

```
# 12-minute system-design answer
1. clarify outcomes and constraints (2 min)
2. define ownership and boundaries (2 min)
3. propose end-to-end workflow (3 min)
4. security, failure, rollback, and evidence (3 min)
5. trade-offs, migration, and metrics (2 min)
```

Production Checklist

- Requirements and nonfunctional constraints are clarified.
- The design names state, identity, inventory, and artifact boundaries.
- Failure, rollback, and recovery are included.
- Trade-offs and rejected alternatives are explicit.
- Claims are supported by portfolio evidence and metrics.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Answer becomes tool list	No organizing requirements or workflow	Start with outcomes, ownership, data flow, and failure model	Interview notes, revised outline

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Candidate over-engineers small scenario	Scale and constraints not clarified	Offer a simple baseline and evolution triggers	Assumption list, staged design
No recovery discussion	Focus remains on happy path	Add partial failure, state loss, rollback, and verification	Failure-mode checklist
Portfolio story lacks credibility	No metrics, constraints, or mistakes	Use specific evidence and explain corrections honestly	Repository, dashboard, incident report

Hands-on Lab 69: Run a Terraform-Ansible System Design Interview Simulation

Prerequisites

- A timer and recording method.
- One architecture prompt and one incident prompt.
- A peer reviewer or scoring rubric.

Implementation

1. Answer a prompt to design automation for 500 environments across three cloud providers and mixed Linux/Windows targets.
2. Spend the first two minutes clarifying scale, ownership, compliance, identity, and recovery.
3. Draw source, policy, state, artifact, inventory, controller, runner, and target flows.
4. Explain partial apply, failed rolling configuration, compromised runner, and state-backend outage.
5. Present a phased migration and measurable SLOs.
6. Review the recording for ambiguity, unsupported claims, missing trade-offs, and excessive jargon; repeat once.

Validation Evidence

- The answer fits the time limit and remains structured.
- At least three trade-offs and three failure modes are explained.
- Ownership and recovery boundaries are visible in the design.
- The second attempt corrects identified weaknesses.

Cleanup

- Delete recordings containing personal or company-sensitive details.
- Retain only sanitized diagrams and scoring notes.

Interview Questions and Model Answers

1. How should you answer an ambiguous automation design question?

Clarify outcomes and constraints, state assumptions, propose a simple baseline, and describe evolution triggers.

2. What differentiates a senior Terraform answer?

State and ownership architecture, recovery, policy, testing, operational workflow, and trade-offs - not only HCL syntax.

3. What differentiates a senior Ansible answer?

Target safety, inventory and variable evidence, idempotency, controller execution context, rolling failure recovery, and content lifecycle.

4. How do you discuss a failed project?

Explain the context, your decision, evidence of failure, corrective action, measured outcome, and what you changed in the engineering process.

Chapter Summary

- Senior interviews evaluate reasoning and communication.
- Clarification precedes architecture.
- Failure and recovery distinguish production designs.
- Credible portfolio stories include evidence and lessons.

Chapter 70: Final Enterprise Automation Factory Capstone

The final capstone integrates the entire Volume 9 curriculum into a production-style automation factory. The solution must provision and configure multiple environments, enforce policy, use keyless identity, produce verifiable artifacts, detect drift, survive partial failure, expose operational SLOs, and pass a controlled recovery game day.

Learning Objectives

- Design and implement an end-to-end Terraform-Ansible automation platform.
- Demonstrate secure delivery, policy, testing, observability, and recovery.
- Operate the platform through failure and drift scenarios.
- Present architecture, evidence, and business outcomes at senior-engineer depth.

Core Concepts

Reference scope: Create development, staging, and production-like environments with separate state, delegated identity, versioned modules, dynamic inventory, controller jobs, policy, and independent verification.

Golden path: A request contract should produce approved infrastructure, a reduced handoff artifact, converged configuration, service verification, evidence manifest, and lifecycle operations.

Security: Use protected source, peer review, pinned dependencies, workload identity, secret manager or controller credentials, signed execution artifacts where available, and least privilege.

Quality gates: Run formatting, validation, tests, plan analysis, policy, Ansible lint and Molecule or equivalent, check mode where meaningful, integration tests, and canary verification.

Operations: Publish dashboards for run health, queue, drift, failures, change lead time, policy results, controller capacity, and target verification. Define incident roles and runbooks.

Graduation evidence: Deliver code, architecture decision records, threat model, service catalog, SLOs, test results, recovery report, cost summary, demo, and retrospective.

Reference Configuration or Command Pattern

```
# capstone acceptance pipeline
request -> validate -> plan -> policy -> approve
      -> apply -> contract -> configure -> verify
      -> evidence_manifest -> promote -> health_assessment

mandatory_failure_tests:
- terraform_partial_apply
- stale_state_lock
- ansible_failed_batch
- compromised_runner_tabletop
- control_plane_restore
- out_of_band_drift
```

Production Checklist

- All environments use isolated state and scoped identities.
- Terraform and Ansible ownership is explicit and non-overlapping.
- The exact approved artifacts are promoted.
- Drift, partial failure, and control-plane recovery are tested.
- A senior reviewer can reproduce the system from documentation.

Troubleshooting Matrix

Symptom	Likely Cause	Corrective Action	Evidence to Retain
Capstone only works on creator laptop	Toolchain, credentials, or paths are implicit	Use pinned execution environments, documented bootstrap, and clean-room test	Rebuild log, artifact digests
Production promotion differs from staging	Pipeline rebuilds or uses floating dependencies	Promote immutable plan/content artifacts and environment data only	Artifact manifest, dependency locks
Recovery requires undocumented steps	Runbook and bootstrap dependencies incomplete	Perform game day with a different operator and update documentation	Game-day timeline, gaps
Demo shows happy path only	Failure and operations evidence omitted	Include drift, partial failure, rollback, dashboards, and postmortem	Incident artifacts, SLO dashboard

Hands-on Lab 70: Build, Break, Recover, and Defend the Automation Factory

Prerequisites

- A Git repository with protected-branch capability.
- Terraform remote state and at least one provider or local virtualization target.
- Ansible execution environment or controller and multiple disposable targets.
- A policy engine or scripted policy gate, artifact storage, and basic observability.

Implementation

1. Write architecture, threat model, ownership matrix, SLOs, request and handoff schemas, and acceptance criteria.
2. Build versioned Terraform modules, Ansible roles or collections, immutable execution artifacts, and isolated environment states.
3. Implement CI for formatting, validation, tests, policy, lint, integration qualification, and evidence generation.
4. Implement progressive delivery from development to staging to production-like targets using scoped workload identity.
5. Inject partial apply, state lock, failed Ansible batch, out-of-band drift, and runner-compromise scenarios; execute documented recovery.
6. Present a clean-room rebuild, operational dashboard, game-day report, cost and value summary, and retrospective to a reviewer.

Validation Evidence

- A second operator can create and retire an environment from documentation.
- Failure tests are detected, contained, recovered, and evidenced.
- Promotion uses immutable reviewed artifacts and separate environment authority.
- SLO, security, cost, and adoption evidence support the final design.

Cleanup

- Destroy all disposable environments through reviewed lifecycle workflows.
- Revoke credentials, remove federation subjects, and archive sanitized graduation evidence.

Interview Questions and Model Answers

1. What is the capstone proving?

That you can operate an automation platform through secure delivery, failure, drift, recovery, governance, and explanation - not just produce code.

2. What is the minimum credible promotion model?

One reviewed source lineage, immutable artifacts, environment-specific authority, policy and approval gates, and independent post-deployment verification.

3. Why require a clean-room rebuild?

It exposes hidden laptop dependencies, undocumented credentials, floating tools, and incomplete bootstrap instructions.

4. What should the final presentation emphasize?

Architecture decisions, measurable outcomes, failure evidence, security controls, trade-offs, and lessons learned.

Chapter Summary

- The capstone integrates every Terraform and Ansible discipline.
- Immutable promotion and scoped identity protect delivery.
- Failure injection and recovery are mandatory graduation evidence.
- Reproducibility and clear explanation demonstrate professional mastery.

Appendix A: Enterprise Troubleshooting Command Reference

Use these commands as investigation tools, not as permission to bypass change control. Preserve state, sanitize logs, and confirm the correct environment before execution.

Area	Command or Technique	Primary Purpose	Safety Note
Terraform	terraform version; terraform providers	Record CLI and provider context	Retain dependency lock file
Terraform	terraform validate -json	Machine-readable configuration validation	Does not validate remote API permissions
Terraform	terraform plan -out=change.tfplan	Create immutable saved plan for review	Protect plan; it can contain sensitive data
Terraform	terraform show -json change.tfplan	Inspect actions and replacement paths	Redact before sharing
Terraform	terraform plan -refresh-only	Investigate drift without proposing configured changes	Review provider reads and permissions
Terraform	terraform state pull	Back up current backend state	Treat output as highly sensitive
Terraform	terraform state list/show	Inspect bindings and attributes	Avoid broad output in shared terminals
Terraform	terraform force-unlock LOCK_ID	Remove confirmed stale lock	Never use until active owner is ruled out
Terraform	moved / removed / import blocks	Reviewable declarative state transition	Prefer over one-time manual commands
Ansible	ansible-config dump -- only-changed	Expose effective nondefault configuration	May reveal paths; sanitize output
Ansible	ansible-inventory --graph / --host	Prove target grouping and variables	Avoid exposing secrets
Ansible	ansible-playbook -- syntax-check	Validate syntax and static structure	Does not prove runtime correctness
Ansible	--check --diff	Preview supported changes	Not every module fully supports check mode
Ansible	-vvv or targeted ANSIBLE_DEBUG	Diagnose connection and execution details	Verbose logs can expose credentials

Area	Command or Technique	Primary Purpose	Safety Note
Ansible	--limit, --tags, --step, --start-at-task	Narrow troubleshooting or recovery scope	Resume only when prior state is understood
Controller	Job events, project sync, inventory sync, instance capacity	Separate controller from playbook failures	Preserve job metadata and image digest
System	DNS, route, TLS, API audit, service logs, metrics	Validate dependencies outside automation tool	Use least privilege and retention policy

Appendix B: Automation Incident Severity and Response Matrix

Severity	Example	Immediate Actions	Exit Criteria
SEV-1	Compromised runner identity; destructive cross-environment change; state loss affecting critical services	Freeze automation, activate incident command, preserve evidence, revoke trust, protect state, notify stakeholders	Trust rebuilt, affected scope reconciled, critical services verified, executive risk accepted
SEV-2	Production partial apply, controller outage, failed multi-host rollout, major drift	Stop promotion, preserve artifacts, contain target set, restore service, perform reviewed recovery	Service stable, state and inventory reconciled, monitoring normal, follow-up owned
SEV-3	Single environment failure, policy false positive, stale inventory, noncritical drift	Limit scope, diagnose, correct through normal review, document defect	Change succeeds, no residual drift, defect tracked
SEV-4	Documentation, warning, performance, or usability issue with no service risk	Record backlog item and evidence	Improvement accepted or explicitly deferred

Appendix C: Volume 9 Graduation Readiness Checklist

Terraform engineering

- ☐ Design remote state, locking, isolation, and recovery.
- ☐ Author typed modules with validation, tests, versioning, and upgrade paths.
- ☐ Import and refactor brownfield resources without unintended replacement.
- ☐ Use plans, policy, drift detection, and evidence safely.
- ☐ Diagnose partial applies, locks, provider failures, and state incidents.

Ansible engineering

- ☐ Build inventories, playbooks, roles, collections, and execution environments.
- ☐ Prove idempotency, variable sources, target selection, and check/diff behavior.
- ☐ Operate controller projects, inventories, credentials, workflows, and capacity.
- ☐ Automate Linux, Windows, network, and application targets appropriately.
- ☐ Recover failed rolling changes and controller incidents.

Integrated delivery

- ☐ Implement explicit Terraform-to-Ansible contracts.
- ☐ Use keyless identity, immutable artifacts, protected source, and policy gates.
- ☐ Promote reviewed artifacts across separated environments.
- ☐ Observe runs, drift, queue, failures, SLOs, and evidence.
- ☐ Run recovery, compromise, and continuity game days.

Professional mastery

- ☐ Explain ownership and architecture trade-offs clearly.
- ☐ Present measurable portfolio outcomes honestly.
- ☐ Use current official certification objectives and ethical study practices.
- ☐ Lead structured troubleshooting and incident communication.
- ☐ Complete the final capstone and clean-room rebuild.

Appendix D: Ten Additional Portfolio-Grade Projects

1. Ephemeral Developer Environment Factory

Create TTL-controlled environments from a service catalog request, with budget limits, policy, automatic verification, and guaranteed cleanup.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

2. Hybrid Linux and Windows Baseline

Provision segmented infrastructure and apply OS-specific hardening, patching, certificate, logging, and compliance evidence.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

3. Network Configuration Assurance

Back up network configurations, apply resource-module changes in serial batches, validate reachability, and detect unauthorized drift.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

4. Secrets and Certificate Rotation Service

Automate issuance, overlapping rotation, consumer verification, revocation, expiry alerts, and evidence across infrastructure and applications.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

5. Database Recovery Qualification Pipeline

Provision isolated restore targets, restore backups, validate consistency and transactions, measure RTO/RPO, and clean up automatically.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

6. Policy Exception Lifecycle Manager

Represent exceptions as code with owner, scope, compensating controls, expiry, alerts, dashboards, and automatic re-evaluation.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

7. Automation Supply-Chain Trust Pipeline

Build execution environments and collections with SBOM, provenance, signing, vulnerability gates, and controlled promotion.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

8. Brownfield Cloud Adoption Factory

Discover unmanaged resources, classify import/rebuild/retire, generate configuration, reconcile drift, and transfer authority safely.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

9. Drift and Continuous Validation Operations

Combine Terraform health assessments or scheduled refresh checks with Ansible compliance verification and incident routing.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

10. Automation Platform Disaster-Recovery Lab

Back up and rebuild state services, controller, registries, projects, and execution artifacts through quarterly game days.

- Required evidence: architecture, threat model, automated tests, run logs, failure exercise, dashboard, cost estimate, cleanup proof, and retrospective.

Appendix E: Ninety-Day Enterprise Automation Practice Plan

Weeks	Focus	Required Deliverable
1-2	Terraform state, modules, import, testing, and recovery refresh	Remote-state lab, tested module, brownfield import, stale-lock drill
3-4	Ansible inventories, roles, execution environments, controller, and troubleshooting	Collection, reproducible EE, controller workflow, failed-batch recovery
5-6	Integrated contracts, CI/CD, workload identity, policy, and artifacts	Keyless pipeline, signed or digested artifacts, policy exceptions
7-8	Enterprise projects: landing zone, platform, stateful service	Three architecture records and production-like labs
9-10	Drift, incident response, performance, and disaster recovery	Drift dashboard, benchmark, two incident drills, control-plane restore
11	Certification objective mapping and timed labs	Objective evidence matrix and two simulations
12	System design, portfolio review, and final capstone defense	Clean-room rebuild, 20-minute presentation, reviewer feedback
13	Remediation and publication	Closed gaps, sanitized portfolio, final readiness checklist

Appendix F: Senior Interview Drill Bank

1. How would you split Terraform state for 1,000 environments owned by 50 teams?
2. How do you recover from a partial apply that created a database but failed before networking completed?
3. When would you use Terraform Stacks, separate workspaces, or an external orchestrator?
4. How do you prevent secrets from entering Terraform state, plans, logs, and Ansible output?
5. How would you import a production environment without causing replacement?
6. Why can an Ansible playbook be non-idempotent even when all tasks report success?
7. How do you prove which variables reached one host in an Automation Controller job?
8. How would you roll out a risky configuration change to 10,000 hosts?
9. What should happen when Terraform succeeds but Ansible configuration fails?

10. How do you investigate a compromised CI runner that had deployment access?
11. How would you design an automation platform for disconnected or regulated environments?
12. Which metrics prove that an automation platform is valuable?
13. How do you govern community modules and collections used in critical environments?
14. How would you test disaster recovery for Terraform state and Ansible controller?
15. What is your most important automation failure, and how did it change your engineering process?

Appendix G: Glossary

Term	Definition
Authoritative owner	The single automation system accountable for the desired state of a mutable property.
Blast radius	The maximum scope of systems, data, or users affected by a failure or unauthorized change.
Brownfield	Existing infrastructure or configuration adopted into a new automation authority.
Change contract	A versioned schema describing inputs, outputs, identities, targets, evidence, and recovery semantics.
Continuous validation	Repeated evaluation that provisioned resources still satisfy declared checks or invariants.
Control plane	Services that authorize, plan, schedule, execute, and record automation.
Drift	Difference between intended configuration, recorded state, and actual remote condition.
Execution environment	A versioned container image containing ansible-core, collections, Python dependencies, and supporting tools.
Golden path	A supported self-service workflow with safe defaults, documentation, and operational ownership.
Idempotency	Property that repeated execution converges without unintended repeated side effects.
Lineage	Terraform state identity used with serial information to protect against unsafe overwrite.
Partial apply	A Terraform run in which some actions complete before a later action fails.
Provenance	Verifiable metadata describing how, where, and from which inputs an artifact was produced.

Term	Definition
Reconciliation	Process of comparing desired, recorded, and actual state and selecting an approved corrective action.
Recovery point objective (RPO)	Maximum acceptable data-loss window measured in time.
Recovery time objective (RTO)	Target duration to restore an acceptable service level after disruption.
Run evidence	Artifacts proving source, identity, approval, execution, target, result, and verification for a change.
State binding	Terraform association between a resource address and a remote object.
Supply chain	Source, dependencies, builders, registries, artifacts, identities, and promotion paths used to deliver automation.
Workload identity	Short-lived federated identity issued to an automation workload instead of a long-lived static secret.

Appendix H: Official Documentation and Certification Index

Verify versions and exam details at the time of use. The links below are official primary sources.

- Terraform documentation: [Open official page](#)
- Terraform state overview: [Open official page](#)
- Terraform state recovery: [Open official page](#)
- Terraform import overview: [Open official page](#)
- Terraform moved block: [Open official page](#)
- Terraform removed block: [Open official page](#)
- HCP Terraform health assessments: [Open official page](#)
- HashiCorp infrastructure automation certifications: [Open official page](#)
- Terraform Associate 004 preparation: [Open official page](#)
- Terraform Authoring and Operations Professional preparation: [Open official page](#)
- Ansible community documentation: [Open official page](#)
- Ansible playbooks guide: [Open official page](#)
- Ansible playbook troubleshooting: [Open official page](#)
- Ansible network troubleshooting: [Open official page](#)
- Red Hat Ansible training and certification: [Open official page](#)
- Red Hat RHCE / EX294 information: [Open official page](#)
- Red Hat Developing Automation / EX374: [Open official page](#)
- Red Hat Managing Automation / EX467: [Open official page](#)

Volume 9 Completion Statement

With Part F complete, Volume 9 now covers Terraform foundations and advanced operations, Ansible foundations and enterprise engineering, integrated secure delivery, and enterprise projects, troubleshooting, certifications, interviews, and final capstone work. A learner who completes the laboratories and evidence requirements should be able to design and operate an automation platform under realistic production constraints.

- Do not claim mastery from reading alone. Complete the build, failure, recovery, and explanation exercises.
- Keep official documentation current and preserve an error journal.
- Publish only sanitized portfolio evidence and respect employer or customer confidentiality.
- Repeat recovery and security game days as tools, providers, platforms, and teams evolve.